

程序设计的实验部分

Bring color to knowledge

Author: 胡潇天 (后简称活性炭)

Institute: 假装有工作室

Date: 2025 年 12 月 26 日

Version: v6.0.0

邮箱: 1985716106@qq.com



人生是一场漫长的旅程，不在乎目的地，而在乎沿途的风景以及看风景的心情。让我们带着希望和勇气，去探索未知的世界，去追求更高的目标

前言部分

实验说明

- 首先十分荣幸作为程序设计的助教来写有关实验部分的文章，对于居于现状的这门课作出课改，整理各类资料，有太多说不出的话语，改革对于一个课程来说不是一时的，需要一届又一届的试验，很感谢支老师能够给我这样一次机会，这次的改革内容将从部分作业的形式上，实验课的内容教学上，课程资料的文档整理上作出实质性的改变。
- 对于书籍的编写，相较于传统的 Markdown 文档局限性，实验部分我采用了 LaTeX 结合二次元的风格进行撰写，让同学们有兴趣去看下去，同时知识内容上采用了一些新的 c++11 之后的新特性去结合实验，而不是原来的古董知识，让同学们能够真正体会到你将来需要什么样的能力，需要什么样的知识。
- 全文实验内容由助教撰写，如有任何代码上或者形式上的纰漏，还请即使通过微信或者邮箱联系助教，当然对于一些蹩脚的英文注释还请多多谅解，我相信大家对于这些内容最后会发出很多很多感慨: ” 这啥啊? ” , ” 这怎么代码写的这么抽象。”, ” 这看也看不懂”，那是因为大家没有习惯过这样的编程方法，这也是求职的时候需要的能力，你只负责你需要完成的部分即可，而且会有一个很长的说明文档需要你去阅读，~~不读然后问助教的是最令人佩服的~~。

致谢：

Vividbook 的开源书籍

vivid 开源书籍将各种带有色彩的文档和代码集于一体，同时带有很多二次元形象

- 官网：https://www.latexstudio.net/index/details/index/mid/3485.html

cs61 系列课程

cs61 系列的课程让我真的燃起了重学计算机的希望，也曾绝望过很多次，也曾彷徨过很多次，焕然一些的课程安排，真的让我意识到了原来计算机真的不必如此枯燥无味。

- cs61a 编程基础官网: https://cs61a.org/
- cs61b 数据结构官网: https://sp24.datastructur.es/
- cs61c 计算机体系结构官网: https://cs61c.org/fa25/

一系列课程

致谢的课程包括但不限于哈佛大学的 cs50 课程，国内的黑马程序员课程，陈越姥姥的数据结构课程，王道的考研课程，斯坦福大学的 cs106L, 课程排名不分先后。

活性炭

2025/09/27 11:01:23



目录

1 lab1 素数分析器 4

1.1 介绍 . . . 4

1.1.1 项目 . . . 4

1.1.2 学习内容 . . . 4

1.1.3 完成内容 . . . 4

1.2 Getting Started . . . 5

1.2.1 Phase 1 . . . 5

1.2.2 Phase 2 . . . 5

1.2.3 Phase 3 . . . 5

1.2.3.1 基于范围的 for 循环 . . . 5

1.2.3.2 使用迭代器进行迭代 . . . 6

1.2.4 Phase 4 . . . 7

1.3 submit . . . 7

1.3.1 提交地点和方式 . . . 7

1.3.2 credit . . . 7

1.4 References . . . 7

2 lab2 AES 加密 8

2.1 介绍 . . . 8

2.1.1 项目 . . . 8

2.1.2 学习内容 . . . 8

2.1.3 完成内容 . . . 8

2.2 Getting Started . . . 9

2.2.1 Phase 1 . . . 9

2.2.2 Phase 2 . . . 9

2.2.3 Phase 3 . . . 9

2.2.4 Phase 4 . . . 9

2.3 submit . . . 10

2.3.1 提交地点和方式 . . . 10

2.3.2 credit . . . 10
2.4 References . . . 10
3 lab3 字符串分析 11
3.1 介绍 . . . 11
3.1.1 项目 . . . 11
3.1.2 学习内容 . . . 11
3.1.3 完成内容 . . . 11
3.2 Getting Started . . . 12
3.2.1 Phase 1 . . . 12
3.2.2 Phase 2 . . . 12
3.2.3 Phase 3 . . . 13
3.2.4 Phase 4 . . . 13
3.3 submit . . . 14
3.3.1 提交地点和方式 . . . 14
3.3.2 credit . . . 14
3.4 References . . . 14
4 lab4 链表的重构 15
4.1 介绍 . . . 15
4.1.1 项目 . . . 15
4.1.2 学习内容 . . . 15
4.1.3 完成内容 . . . 15
4.1.4 链表介绍 . . . 16
4.2 Getting Started . . . 16
4.2.1 Phase 1 . . . 16
4.2.2 Phase 2 . . . 18
4.2.3 Phase 3 . . . 19
4.3 submit . . . 19
4.3.1 提交地点和方式 . . . 19
4.3.2 credit . . . 19
4.4 References . . . 19
5 lab5 历史管理 20
5.1 介绍 . . . 20
5.1.1 项目 . . . 20
5.1.2 学习内容 . . . 20
5.1.3 完成内容 . . . 20
5.1.4 前置内容 . . . 21
5.2 Getting Started . . . 21
5.2.1 Phase 1 . . . 21
5.2.2 Phase 2 . . . 21
5.2.3 Phase 3 . . . 21
5.2.4 Phase 4 . . . 21
5.2.5 Phase 5 . . . 22

5.2.6 Phase 6 . . . 22
5.2.7 Phase 7 . . . 23
5.3 submit . . . 23
5.3.1 提交地点和方式 . . . 23
5.3.2 credit . . . 23
5.4 References . . . 23
6 lab6 2048 24
6.1 介绍 . . . 24
6.1.1 项目 . . . 24
6.1.2 学习内容 . . . 24
6.1.3 完成内容 . . . 24
6.1.4 前置内容 . . . 25
6.2 Getting Started . . . 25
6.2.1 Phase 1 . . . 25
6.2.1.1 横向移动 . . . 25
6.2.1.2 压缩合成 . . . 25
6.2.1.3 再次压缩 . . . 25
6.2.1.4 搬运 . . . 25
6.2.2 Phase 2 . . . 26
6.2.3 Phase 3 . . . 26
6.3 submit . . . 27
6.3.1 提交地点和方式 . . . 27
6.3.2 credit . . . 27
6.4 References . . . 27
7 lab7 解释器 28
7.1 介绍 . . . 28
7.1.1 项目 . . . 28
7.1.2 学习内容 . . . 29
7.1.3 完成内容 . . . 29
7.1.4 前置内容 . . . 29
7.2 Getting Started . . . 29
7.2.1 解释器运作图 . . . 30
7.2.2 Phase 1 . . . 30
7.2.3 Phase 2 . . . 31
7.2.4 Phase 3 . . . 32
7.2.5 Phase 4 . . . 34
7.2.6 Phase 5 . . . 37
7.3 submit . . . 37
7.3.1 提交地点和方式 . . . 37
7.3.2 credit . . . 37
7.4 References . . . 37



第 1 章 lab1 素数分析器

❤无畏孤单，因为这个世界上，肯定有一个人正努力的走向你.❤

——《龙猫》

1.1 介绍

在本实验中，你将创建一个素数分布器，一个素数分布器可以有多重多样的功能，比如判断一个数是否是素数，寻找 1 到 n 中的所有素数，输出素数之和等功能

1.1.1 项目

下载 lab1.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug
- CMakeLists.txt
- Lab1.cpp

1.1.2 学习内容

lab1 的学习内容较 lab0 较为庞大，知识量也较为多，请注意不要投机取巧采用一些简单的方式进行运算，而是强迫自己使用文档想要教给你的东西

- 素数的判断以及优化
- 如何使用动态数组 vector 的一些基本知识
- 如何使用迭代器以及基于范围的 for 循环
- 如何合理的使用头文件中的各种强大函数

1.1.3 完成内容

逐步 is_prime(),memory_prime,iteration(),sum() 等多个函数

1.2 Getting Started

1.2.1 Phase 1

执行 is_prime, 这个函数能够帮助我们去判断一个数是否是素数, 他需要一个参数:

- x: 传入一个任意的自然数

is_prime 函数，若一个数为素数则返回 true，若一个数是合数或者既不是质数也不是合数则返回 false

Axiom 1.1 (Hint)

- 对于一个数 n 来说，我们需要从 2 开始判断是否能够整除该数，这个数如果能够整除 n，则存在因子，不是素数。
- 从某种意义上将，我们只需要遍历到 $\sqrt{n}$ 即可，不需要遍历到 n，可以想想为什么。

1.2.2 Phase 2

执行 memory_prime 函数，能够将 1 到 n 中所有的素数存储在一个叫 primes 的数组中，但是根据常规的方法，你可以想到，假设我这个 n 是非常大的，int 以内的素数大约有一亿个，我需要开一个这么大的数组，但如果我的 n 很小，又会导致我开的一亿大小的数组被浪费掉了很多空间，所以我们这里引入动态数组的概念。

Definition 1.1 (vector)

vector 是 *C++* 标准模板库（*STL*）中的动态数组。和普通数组不同，*vector* 的长度可以动态增长或缩小。内存是连续分配的，所以可以像数组一样通过下标访问元素。

你可能需要以下知识:

```
vector<int> primes; //初始化一个动态数组,叫primes
int num ; cin >> num ;
primes.push_back(num); //向primes中插入一个num.
```

该函数有两个参数:

- vector<int>primes, 传入一个你需要修改的动态数组，向其中插入所有的素数
- n 你需要插入的范围的上确界。

1.2.3 Phase 3

1.2.3.1 基于范围的 for 循环

执行 iteration 函数，但是你发现有两个这样类型的函数，但是一个后面使用了 for 循环，这里我们的实验要求使用一种特殊的 for 循环，称为基于范围的 for 循环，用于遍历其中的元素。可以理解成从一个数组中取出一个值，然后对其操作。

你可能需要以下的代码形式:

```
for(const int & x : v)
{
    cout << x ;
}
```

使用冒号来分割，可以理解成 v 中的元素取出来到了 x 里面，然后完成输出的操作之后，自动的进行下一个数的操作，继续输出，但是这里注意的是 const 和 & 符号，其中 const 确保了里面的值不会被修改，而 & 是需要修改 v 中值的时候用的。~~这不左右脑互相博弈吗?~~ 这里的 & 起到的作用是防止在正常遍历过程中产生大量的副本，其实就是值传递的本质，而引用传递会确保不会产生副本加快程序的运行速度。

1.2.3.2 使用迭代器进行迭代

对于第二个版本的迭代你需要一个叫迭代器的功能来实现，说白了就是指向第一个数的指针

Definition 1.2 (迭代器)

迭代器（*iterator*）是 *STL*（标准模板库）中的一个“泛化指针”。它的作用是用统一的方式访问容器里的元素，而不用关心容器底层是数组、链表、树还是哈希表。可以把迭代器理解为指向容器中元素的对象。

请参考一下代码进行迭代器的循环使用

```
#include <vector>
#include <iostream>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 4};
    // 定义迭代器
    vector<int>::iterator it;
    // 让迭代器指向容器开头
    it = v.begin();
    cout << *it << endl; // 输出 1
    it++;
    cout << *it << endl; // 输出2

    it = v.end();
    cout << *it << endl; // 输出4
}
```

ps: 最终输出格式为 1 2 3 4 5 这样，中间用空格隔开然后占用一行。

1.2.4 Phase 4

实现 sum 函数，我想这个时候就有很多人要说了，sum 函数遍历数组即可实现，这里我们采用比较现代化的实现，c++ 库中有非常多好用的多函数，我们的实验部分不需要大家去思考很多高级的算法，只需要去知道有这个东西，毕竟实际开发中是不需要写很多已有的东西的。

- 首先请在头部写好这个函数需要的头文件 #include <numeric>

请使用函数 accumulate(first,end,init) 来实现求和的功能，该函数有三个参数第一个参数是指向数组头部的迭代器，第二个是指向尾部的迭代器，第三个是求和的初始值。

1.3 submit

1.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，只需要提交 cpp 后缀的文件即可，其他的文件不需要提交。

1.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

1.4 References

- 有关 vector 的一些使用
- 基于范围的 for 循环使用



第 2 章 lab2 AES 加密

❤无畏孤单，因为这个世界上，肯定有一个人正努力的走向你.❤

——《龙猫》

2.1 介绍

AES（Advanced Encryption Standard，高级加密标准）是一种对称分组加密算法，目前广泛应用于文件加密、通信加密（如 HTTPS、VPN）、无线网络安全（如 WPA2/WPA3）等场景, 但由于其具体实现原理复杂，我们只对明文做部分的加密.

2.1.1 项目

下载 lab2.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug
- CMakeLists.txt
- Lab2.cpp

2.1.2 学习内容

lab2 的更多操作作用在数组上，学习一些有关数组的简单替换操作

- 一维数组转二维数组
- 数组的移动
- 二进制异或
- 二维 vector 的使用

2.1.3 完成内容

逐步 marice(),xor_key(),replace_value(),move_matrix(),shiftleft() 等多个函数

2.2 Getting Started

在输入时你将得到一个字符串，请将其用名为 v_str 的动态数组将其存储. **请注意**从 str 中取出的是 char 字符，需要转成其 ascii 码。例如: 输入:sdjk 数组内存的是:115 100 106 107

2.2.1 Phase 1

执行 marice(), 这个函数能将我们的矩阵转成二维的，这里使用 vector<vector<int» 的嵌套形式，仔细想想普通二维数组的使用方法这个函数有一个参数

- v: 传入的 int 数组

AES 加密的数组是非常奇特的，他不是正常的排序方式，而是下面的方式

$$\begin{bmatrix} 1 & 2 & 3 & 4 & 5 & 6 & 7 & 8 & 9 \end{bmatrix} \longrightarrow \begin{bmatrix} 1 & 4 & 7 \\ 2 & 5 & 8 \\ 3 & 6 & 9 \end{bmatrix}$$

2.2.2 Phase 2

执行 xor_key 函数，将 str 这个明文秘钥和 key 这个系统给出的秘钥进行相应位置的二进制操作 (即异或)，测试数据中没有给出任何的两个二维数组大小不相同的情况，请放心的对相应位置进行异或操作。最终返回一个新的异或数组该函数的参数列表:

- a : 明文秘钥的 ascii 二维数组
- b : key 秘钥的 ascii 二维数组

2.2.3 Phase 3

执行 replace_value() , 该函数的具体作用主要是对照 S_BOX 表，将获得的异或矩阵根据低八位进行替换，用一点专业的术语就是，使用最低八位的二进制数字映射成 S_BOX 表中的新十六进制数这个映射的思想叫做哈希函数。

Definition 2.1 (哈希函数)

哈希函数是一种把任意长度的输入（关键字、数据）映射为固定长度输出（哈希值、散列值）的函数。

该函数有一个参数:

- v: 一个异或之后的二维数组.

由于我们本题的 S_BOX 表使用的是 256 个数的，所以请一定要限制自己的矩阵每个数在 0 到 255 的范围，这个操作可以通过一个数与 0XFF 取与来进行.

2.2.4 Phase 4

执行 move_matrix() 函数, 将二维数组中的第 i 行数向左移动 i 个位置，请时刻记住，移动操作是一个可以抽象出来的东西，这也是 c++ 最重要的抽象思想，在移动过程会使用 shiftleft 函数，而不是写一大堆的 for 循环。

Definition 2.2 (抽象)

在 C++（以及面向对象编程）中，抽象（*Abstraction*）是指把对象的共性和本质特征提取出来，忽略不相关的细节，只关注对外提供的功能和行为。

1. move_matrix 函数有一个参数:
- v: 传入的需要移动的二维数组

2. shiftleft 函数有两个参数:
- row: 传入需要移动的每行数组
- step: 需要移动的位数

提升版本：如果觉得正常的移动难不倒你，请使用 algorithm 中的 rotate 函数进行移动, 其中第一个参数是指向头部的迭代器，第二个参数是指向新头部的迭代器，第三个参数是指向尾部的迭代器，如果使用的话请参考下面这段代码

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
int main() {
    std::vector<int> v = {1, 2, 3, 4, 5};

    // 将前两位元素移动到末尾
    std::rotate(v.begin(), v.begin() + 2, v.end());

    for (int x : v) {
        std::cout << x << "␣"; // 输出3 4 5 1 2
    }
}
```

2.3 submit

2.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，只需要提交 cpp 后缀的文件即可，其他的文件不需要提交。

2.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

2.4 References

- AES 加密的实现流程
- 二维 vector 数组的使用技巧



第 3 章 lab3 字符串分析

❤无畏孤单，因为这个世界上，肯定有一个人正努力的走向你.❤

——《龙猫》

3.1 介绍

在正常进行 C++ 开发中，我们是需要用到很多有关读写文件的操作的，比如老板让你读入一个 txt 文件，然后把里面所有有效数据提取出来 (更明显的特征还是 python 中的 panda 等一系列库)

3.1.1 项目

下载 lab3.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug
- CMakeLists.txt
- Lab3.cpp
- text.txt

3.1.2 学习内容

lab3 的操作大部分在于文件读写和字符串的一些操作上。

- 如何通过流运算读入文件
- 如何使用哈希表
- 使用常见的字符串内置函数

3.1.3 完成内容

逐步实现读入文件、分割字符串、统计字符数、对字符串进行变换等操作。

3.2 Getting Started

3.2.1 Phase 1

在 main() 函数中读入文件 test.txt, 你可能需要的以下的操作 (所有的-> 都解释了需要什么样子的变量类型, 在 c++ 中称为尾置类型推导，不用自己写，提醒一下你需要什么类型的变量):

```
ifstream 变量名字(文件名字->string);
string str ;
while(getline(存文件的变量->ifstream ,str ))
{
    istringstream 变量名字(str) ; //定义字符串流
    string word ; //开始将一行按照空格分割单词
    while(变量名字->istringstream >> word)
    {
    } // 流给单个单词
}
变量名字->ifstream.close() //记得关掉文件
```

此时的问题又来了, 如何记录每次单词的数量呢? 是否还记得我们 Lab2 中提到的哈希函数，将一个东西映射到另一个东西 C++ 中也封装了这样的快捷方式，叫做 Map<key,value>, 其中 <> 类似于 vector 的类型填写，只不过现在有类型转换，需要两个类型。

```
#include <map>
map<int,int> mp ;
mp[3] ++ ;
mp[4] ++ ;
map<char,string>ma ;
ma['a'] = "hello world" ;
cout << ma.first << " " << ma.second << endl; //输出a hello wor;d ;
```

这样的操作使得我们能够将两种类型相互映射，第一个为 key, 第二个称为 value

now，赶快去完成你的第一个阶段，读写文件并统计每个单词的数量吧。

3.2.2 Phase 2

既然我们已经知道了如此多的单词极其数量，接下来肯定是对其进行分析，请找出数量最多的单词，变量的使用请根据前后文自行判断, 有些输入已经做好的。

现在问题来了，如何遍历一个 Map 呢？那你是否对之前的 vector 熟悉呢？举一反三，是否能够通过迭代器和 for 遍历来进行操作呢？答案是肯定的，由于我们这里是遍历所有的，所有我们采用后者，但是问题又来了，我们从 map 中取出来的每个元素是什么呢？vector 取出来的都是 int，double 这些常用数据类型这里进一步体现了 c++11 的便捷操作，auto 关键字，不需要在意中间变量的类型, 只需要用 auto 进行自动类型推导即可。

```
    for(auto c : mp){
        cout << c.first << " " << c.second << endl ;
    }
```

这样我们巧妙的跳过了中间变量 c 的类型，成功的使用了 first 和 second.

接下来请使用 for 遍历的方式，遍历所有“中间变量”，来找到最大的单词和数量吧。

3.2.3 Phase 3

将最大数量的单词字符串翻转，并且统计每个字母的个数

翻转过程推荐两个操作:

1. 调用 algorithm 中的 reverse() 函数,reverse(BidirIt first, BidirIt last) 其中该函数有两个参数，第一个为头指针，第二个为尾指针，获取方式同 vector

2. 使用双指针的方式，i 指向字符串头，j 指向字符串尾, 交换两个指向的具体值，然后两个指针向中间靠拢。

3.2.4 Phase 4

现在，请将已经翻转后的单词全部大写，并且移除出现最多的字母 (你已经统计过了)，为了方便我们的操作，我们需要使用一些 c++ 库中的东西, 请参考以下代码

```
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;

int main() {
    vector<int> v = {1, 2, 3, 2, 4, 2, 5};
    auto newEnd = remove(v.begin(), v.end(), 2); 去掉所有的2
    cout << "after remove: ";
    for (int x : v) cout << x << " "; // 1 3 4 5(迭代器指向这里) 2 2 2
    cout << "newEnd 指向索引 " << (newEnd - v.begin()) << endl; //输出5
    return 0;
}
```

字符串也同理，remove 只能将所有的某个字母移到最后，然后返回新的迭代器位置.

此时还需要一步, 用 erase 函数来去掉末尾的几个重复的数字

```cpp
#include <iostream>
#include <algorithm>
#include <vector>
using namespace std;
int main() {
    vector<int> v = {1, 2, 3, 2, 4, 2, 5};
    auto newEnd = remove(v.begin(), v.end(), 2); //去掉所有的2
    cout << "after remove: ";
    for (int x : v) cout << x << " "; // 1 3 4 5(迭代器指向这里) 2 2 2
    cout << "newEnd 指向索引 " << (newEnd - v.begin()) << endl; //输出5
    v.erase(newEnd , v.end() ); // 去掉末尾的数
    return 0;
}
```

这个实验的知识点就全部讲到这里了，需要能多多理解，不会的可以参考 refercences

3.3 submit

3.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，需要提交 cpp 后缀的文件以及 txt(打包为压缩包)，其他的文件不需要提交。

3.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

3.4 References

- C++ Map 的使用
- 字符串使用技巧
- 流读写文件



第 4 章 lab4 链表的重构

❤无畏孤单，因为这个世界上，肯定有一个人正努力的走向你.❤

——《龙猫》

4.1 介绍

指针是 c++ 中一个老生常谈的话题，管理不好可能会有内存泄漏的风险，比如不及时释放指针，但也因为诞生了很多著名的数据结构，比如链表、二叉树等，本实验将着重构造一个能够稳定运行的双向链表代码。

4.1.1 项目

下载 lab4.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug
- CMakeLists.txt
- Lab4.cpp
- Lab4demo.cpp

4.1.2 学习内容

- 理解指针的含义
- 学会使用单链表的基本操作
- 对比基本的时间复杂度

4.1.3 完成内容

根据已有的无头结点的链表，逐步实现一个双链表的插入，删除，遍历操作。

4.1.4 链表介绍

链表（Linked List）是一种常见的基础数据结构，是一种线性表，但是并不会按线性的顺序存储数据，而是在每一个节点里存到下一个节点的指针（Pointer）。链表是由数据域和指针域两部分组成的，它的组成结构如下：

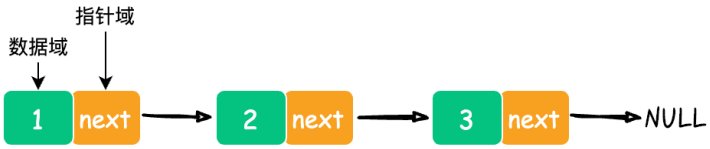


图 4.1: 链表结构图

4.2 Getting Started

4.2.1 Phase 1

此阶段我们先阅读 demo 文件中的代码 (读代码也是程序员的基本能力), 如果有不会的需要及时提问，或者上网查询。

```
struct Node{
    int data;
    Node* next;
    Node(int val = 0) : data(val), next(nullptr) {}
};
```

上方这段代码用结构体写了一个简单的节点，int data 相信不用多说

- 有一个新的变量，叫做 Node*, 他的作用就是帮助我们指向下一个节点域，我们可以通过 Node->next 这样的操作来访问下一个节点，那么 Node->next->data 就是下一个节点的具体值, 这就是指针的本质，可以存放下一个节点的地址，然后通过寻址访问。
- 还有一个新的语法是 Node(), 直接以结构体的名字来命名的函数，我们将其称为构造函数，可以通过在创建变量时，让程序帮我们去初始化我们需要的变量。
- Node():data(val) 意味着可以将 val 赋值给 data , 等价于 Node() {data = val;}
- nullptr: C++11 引入的新关键字能够将一个指针指向空，避免野指针的出现。

```
Node* createList() {
    Node* head = new Node(); // 带头结点
    head->next = nullptr;
    return head;
}
```

上方这段代码是需要我们先去创建一个空的头结点: 创建后生成的是 (head)[val = 0| nullptr] -> [???] 返回我们头结点所在的位置。

Definition 4.1 (头结点和哨兵节点)

一般来说，头结点和哨兵节点差不多，都是为了防止出现头部为空，然后需要判断情况而设置的一个虚拟节点。

将头结点的位置随便设置一个值, 然后每次添加的时候从这个节点的下一个位置开始添加，这样会省不少事情，否则我每次还得判断当前位置是否为头结点，如果自己去写一个这样的链表会发现有很多的情况要处理: 比如删除节点，我得判断删的是不是第一个节点，但是如果引入虚拟哨兵节点，直接删掉下一个节点就可以了

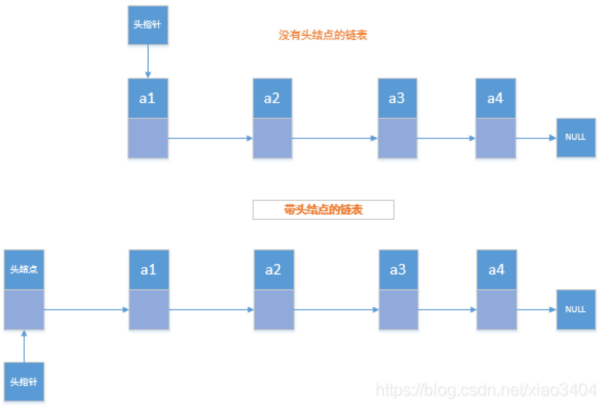


图 4.2: 链表结构图

创建结束之后我们需要往头结点之后插入值。如何实现插入呢? 链表是不支持随机位置插入的，我们需要一步一步走，直到走到 next 为 nullptr 的节点位置，然后将 val 赋值。

```
// 尾插法建立链表
void buildListTail(Node* head, int n) {
    Node* tail = head;
    for (int i = 0; i < n; i++) {
        int val;
        cin >> val;
        Node* newNode = new Node(val); // 创建一个节点，值为val
        tail->next = newNode;
        tail = newNode;// tail = tail->next
    }
    tail->next = nullptr;
}
```

上面的代码可能会有下面的疑惑

- 为什么需要 tail 指针, 请时刻记住 head 指针是永远不能移动的，否则会丢失，地址的遍历不会像数组一样，数组想在哪个位置遍历就在哪个位置遍历，因为地址就在那里定死了，然后指针不一样，一旦动了头指针就找不到了所以需要一个 tail 来跟踪每个位置，让 tail 往后移动就可以了。
- 如何加节点，首先得创建一个值为 val 的节点，然后让 tail 的 next 变成我们新节点，因为此时 tail 是

前一个节点，然后 tail 往后移动就可以了，这里可以使用 tail = newNode , 也可以使用 tail=tail->next ;

如果这里没有理解，请不要迅速往下看，请搞懂指针的指向关系。接下来，我们需要的是将链表中的一个元素删除

```
void deleteNode(Node* head, int val) {
    Node* p = head;
    while (p->next && p->next->data != val)
        p = p->next;
    if (p->next) {
        Node* tmp = p->next;
        p->next = tmp->next;
        delete tmp;
    }
}
```

阅读完上面的代码，会发现删除的遍历过程同增加元素的过程，都需要先遍历到相应的元素，然后再做处理。通过将一个点的 next 指向其 next 的 next。（仔细理解这里）

Axiom 4.1 (Hint)

- 删除元素的时候，需要将一个点的 *next* 指向 *next* 的 *next*，是名义上的删除了这个点，但这个点仍存在于物理内存中，所以需要释放掉这个点的指针

如果你成功理解了上面的方式，那么遍历链表也不在话下了，只是一直在调整指向关系。

```
void printList(Node* head) {
    Node* p = head->next;
    while (p) {
        cout << p->data << " ";
        p = p->next;
    }
    cout << endl;
}
```

4.2.2 Phase 2

本阶段我们将在理解单链表的基础上，进一步构建一个功能更完整的数据结构：双向链表（Doubly Linked List）。

与单链表相比，双向链表的每个节点不仅保存下一个节点的地址，还需要保存上一个节点的地址。这也意味着我们在插入、删除节点时，不仅要修改 next 指针，还需要同步维护 prev 指针，使整个链表保持一致性。

仿照单链表的行为，完成双向链表的基本操作，包括：

- **创建链表**：构建一个带头结点的双向链表，并保证头结点仅作为哨兵节点，不存储有效数据。同时维护一个尾指针，使得尾部插入可以在 $O(1)$ 时间内完成。

- **使用尾插法建立链表：**依次生成若干个节点并追加到链表末尾。过程中需要更新新节点的 prev 指针、原尾节点的 next 指针，以及整体的 tail 位置。
- **实现插入：**在指定节点的前方或后方插入新节点，并正确修改四条相关指针。需要考虑插入在链表头部与尾部时的特殊情况。
- **实现删除：**删除任意节点时，通过调整其前驱与后继节点的指针完成逻辑删除，并使用 delete 释放被删节点的内存。若删除的是尾节点，记得更新 tail。
- **双向遍历：**使用 next 指针从前向后遍历链表，使用 prev 指针从后向前遍历链表。双向链表的优势就在于反向遍历无需寻找末尾位置。
- **释放链表：**最终需要释放链表中的全部节点（包括头结点），确保不产生任何内存泄漏。

4.2.3 Phase 3

这个阶段的内容主要是，对比一下内置的 list 和自己编写的 list 的区别，请保证你的所有实现已经正确的基础上，uncomment 掉 main 函数中的 speedTest() , 重新运行即可会出现一个对比的过程, 如果不能运行，可能是由于自己的 C++ 版本不够高，所以请确保你的 C++ 版本足够，如果不能运行，可以直接跳过本过程。

ps: 提交到 cg 的时候请把这个操作的代码注释掉

```
[Speed Test with 200000 random numbers]
MyList push_back: 1.4612 ms
std::list push_back: 14.9652 ms
```

图 4.3: 运行图

这个实验的知识点就全部讲到这里了，需要能多多理解，不会的可以参考 refercences

4.3 submit

4.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，需要提交 Lab4.cpp 文件, 其他的文件不需要提交

4.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

4.4 References

- C++ 单链表的实现
- 带头结点和不带头结点的区别



第 5 章 lab5 历史管理

❤无畏孤单，因为这个世界上，肯定有一个人正努力的走向你.❤

——《龙猫》

5.1 介绍

在本实验中你需要像你的浏览器一样，实现对于已经访问信息的记录与统计。

5.1.1 项目

下载 lab5.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug
- CMakeLists.txt
- Lab5.cpp
- HistoryManager.cpp
- HistoryManager.h

5.1.2 学习内容

lab5 的学习的内容大致可以算前面内容的运用，同时引入类与对象，学完这一版块，可以算是正式进入了 STL 时代。

- string 内置函数的使用
- 哈希表的统计使用
- 动态数组的使用
- 常用的算法库函数

请坚信你能完成，因为你都用过，只是不熟练而已

5.1.3 完成内容

实现三个基础功能和四大扩展功能。

5.1.4 前置内容

一点变量名字的使用:

- recentHistory: 存放最近使用的网址
- visitcounts: 存放网址的访问次数
- favorites: 存放我收藏的网址

5.2 Getting Started

5.2.1 Phase 1

实现 visit() 函数，这个函数有一个参数。

- url: 传入的网址

这个函数的作用是表示我最近访问了一个 url 网址，然后访问次数需要加 1。

代码长度: 2-3 lines

5.2.2 Phase 2

实现 addToFavorites() 函数，这个函数有一个参数。

- url: 最喜欢的网址

这个函数的目的是，我找到了一个网址，我想要将其加入到我收藏的列表中。

代码长度: 1 lines

5.2.3 Phase 3

实现 clearHistory() 函数，该函数有一个参数

- url: 我要移除的最近访问网址

这个函数的作用是我要移除我历史访问记录中的某个网址，记得移除访问过的数量

代码长度: 2-3 lines

基础功能完

至此基础功能已经结束，基础功能部分只是为了回顾一下一些常用的操作，比如哈希表的插入，动态数组的插入，以及删除等操作。

5.2.4 Phase 4

实现 getMostVisited() 函数，该函数无参数

该函数的目的是获取访问最高的网址，如果访问量均为空，请返回”None”, 否则返回访问量最高的网址字符串。

Axiom 5.1 (Hint)

- 哈希表最快捷的遍历莫过于基于 *for* 循环的遍历，不会的话请参考实验一。

有效代码长度: 8-10 lines

5.2.5 Phase 5

实现 deduplicateFavorites() 函数，该函数无参数

该函数的目的是将我收藏的网址信息去重, 你需要首先将收藏夹中的网址按字典序排序，然后去重。

Axiom 5.2 (Hint)

- 排序最快捷的方式是 *sort()* 函数
- 去重最快捷的函数是 *unique* 配合 *erase* 函数（回实验三看看 *erase*）

但请记住，去重一定要保证是有序的，所以用 sort(头位置, 尾位置) 来对容器排序，对于存字符串的一般按照字典序来进行排序，然后使用 unique 和 erase 去重, 请参考下面代码来使用。

```
#include <iostream>
#include <algorithm>
#include <vector>

int main() {
    std::vector<int> v = {1, 1, 2, 2, 2, 3, 4, 4, 5};

    // 移除连续重复元素
    auto last = std::unique(v.begin(), v.end());

    // 注意：容器大小没有改变！
    std::cout << "容器大小: " << v.size() << std::endl; // 输出: 9

    // 只处理到 last 之前的位置
    v.erase(last, v.end());  // 真正删除重复元素

    std::cout << "去重后: ";
    for (int x : v) std::cout << x << " "; // 输出: 1 2 3 4 5
    std::cout << std::endl;

    return 0;
}
```

有效代码长度: 5 lines

5.2.6 Phase 6

实现 searchByKeyword() 函数，该函数有一个参数

- key: 我要查询的关键词

该函数的目的是在最近访问网址中，看看有没有网址内存在关键词 key.

```
std::cout << "[Match]: " << url << std::endl;
```

Axiom 5.3 (Hint)

- 既可以使用直接遍历也可以使用基于范围的 *for* 循环遍历
- 字符串的查找用 *find* 函数最方便

有效代码长度: 3-5 lines

5.2.7 Phase 7

实现 undoLastVisit() 函数，该函数无参数
该函数的目的是撤销最近的一次访问操作, 然后输出语句如下

```
std::cout << "Undid visit to: " << last << std::endl;
```

请确保你同时清除了最近访问的元素以及其访问次数.

Axiom 5.4 (Hint)

- *back()* 可以访问一个动态数组的最后一个元素

有效代码长度: 5-8 lines

5.3 submit

5.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，需要提交所有 cpp 和 h 后缀的文件，其他的文件不需要提交。

5.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

5.4 References

- 请参考实验 1-4 的所有参考文献来完成本实验。



第 6 章 lab6 2048

❤无畏孤单，因为这个世界上，肯定有一个人正努力的走向你.❤

——《龙猫》

6.1 介绍

2048 是一款风靡全球的游戏，而且诞生了很多别的同类型合成游戏, 他的唯一目的就是通过上下左右键操作，来最终合成 2048

6.1.1 项目

下载 lab6.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug
- CMakeLists.txt
- Lab6.cpp

6.1.2 学习内容

lab6 的实验可能没有前面那样想让你刻意的去使用 STL 中的很多容器，他的目的反而是让你去体会一下面向对象的小型开发，更多的考验你的思维，而且不用记忆那么多复杂的函数

- 学会对数组进行合并
- 学会移动数组

6.1.3 完成内容

对于 2048 而言其实，就三个操作，按 WASD 来移动砖块，然后合并砖块，判断游戏结束。

6.1.4 前置内容

一点变量名字的使用:

- board[4][4] 整个游戏的 4*4 砖块
- score 目前的得分

开始前说明：请不要过多的关注于整个游戏是如何实现的，而是只聚焦你要实现的部分。

6.2 Getting Started

6.2.1 Phase 1

实现 bool compressAndMerge(), 该函数有一个参数

- row[4]: 从期盼中取出的一行/一列四个元素。

这个函数的作用是合并砖块，然后返回是否发生了合并。合并可以一步一步来进行

6.2.1.1 横向移动

1. 将所有非 0 数字按原顺序移到最左边

效果：$[2,0,0,2] \longrightarrow [2,2,0,0]$

Axiom 6.1 (Hint)

- *需要开一个长度为 4 的数组，然后将所有非 0 元素暂存起来*

6.2.1.2 压缩合成

2. 将相同元素合并起来效果：$[2,2,4,4] \longrightarrow [4,0,8,0]$

Axiom 6.2 (Hint)

- *增加得分（2048 规则：得分等于新生成的方块值）*
- *将后位清零，防止在同一次移动中发生链式合并*

6.2.1.3 再次压缩

3. 移除步骤 2 合并过程中产生的“0”间隙

效果：$[4,0,8,0] \longrightarrow [4,8,0,0]$

6.2.1.4 搬运

4. 我们整个过程如果运用了临时数组来处理，那么还需要搬运一步来完成从临时数组到 row 数组的搬运，如果只是在原数组上进行操作，那么可以不用这一步。

友情提示

如果只是将代码塞给 AI 的话是体会不到整个设计游戏的快乐过程的，助教认为初上手游戏制作快的可能需要三四个小时，慢的可能需要七八个小时来完成。听从自己的本心去写代码，不论结局如果，自己写的一定是最具有价值的。

6.2.2 Phase 2

实现 move() 函数，该函数有两个参数

- direction: 移动的方向, 传入 WASD 四个方向
- addRandom: 无需在意该参数，只是为了测评用

该函数就是需要实现上下左右移动了, 实现流程大致如下

1. 将传入的方向键小写
2. 判断四个方向
3. 然后遍历每行/每列，依次通过之前 Phase 1 的合并规则，合并每行/每列
4. 修改整个 board 图。

为了降低大家的难度，这里提供一个方向的参考代码

```
if (direction == 'd') {
        for (int i = 0; i < 4; ++i) {
            int row[4] = {board[i][3], board[i][2], board[i][1], board[i][0]};
            if (compressAndMerge(row)) {
                changed = true;
                board[i][0] = row[3]; board[i][1] = row[2];
                board[i][2] = row[1]; board[i][3] = row[0];
            }
        }
}
```

由于我们的设计仍存在发展阶段，这里可以提供一种思路就是将上下右四个方向映射成向左的方向，只是看的方向不一样，但是逻辑是相同的, 当然你写成四个 if 语句也没有问题。

如果能这样写出来代码 (前提是自己写的) 可以提供一部分小小的加分，鼓励大家的创新思维。

6.2.3 Phase 3

实现 checkGameOver() 函数，该函数无参数

这个函数用来判断游戏是否结束，这个函数的情况有很多，场上还存在空白方块（值为 0），存在可合成方块（相邻相同），此部分就是开放式的发挥了，比如有人就不想玩到 2048 结束，当然可以玩到 4096。自行发挥即可

友情提示

接下来就是玩的阶段了，就请主函数中将模式切换到 play(), 提交代码的时候切换到 test(), 由于代码的一些局限性，测试部分主要测试大家的合成和移动规则，结束规则可以自行拟定。

最终效果如下图

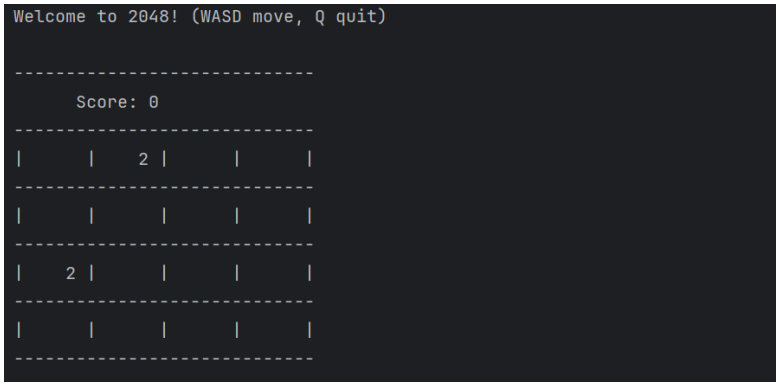


图 6.1: 游戏运行图

6.3 submit

6.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，提交到 cpp 后缀的文件，同时将模式切换到 test().

6.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

6.4 References

- 由于很多版本是 java 和 python 的，C++ 写的很少，而且原理差距很大，建议参考文档



第 7 章 lab7 解释器

❤不管前方的路有多苦，只要走的方向正确，不管多么崎岖不平，都比站在原地更接近幸福。❤

——《千与千寻》

7.1 介绍

项目开始前我们对比一下不同语言底层的处理机制

对比维度	解释器（Interpreter）	编译器（Compiler）
执行方式	逐行（或逐语句）分析并立即执行	先整体翻译，再执行生成的程序
是否生成可执行文件	否	是
执行流程	源代码 → 分析 → 执行	源代码 → 编译 → 可执行文件 → 运行
错误发现时机	运行到对应语句时才发现	编译阶段即可发现大部分错误
运行效率	相对较慢（需反复解析）	较快（已翻译为目标代码）
平台相关性	跨平台性较好	与平台、体系结构相关
典型语言	Python、JavaScript、Ruby	C、C++、Rust
调试体验	方便，修改后可立即运行	需要重新编译后运行
实现复杂度	相对简单	复杂（代码生成、优化）

表 7.1: 解释器与编译器的对比

其实概括来说，解释器就是一句语句一句语句看，直到运行到有错误的地方; 编译器就是一下全部看，有错误就跑不起来。而 Python 语言的解释器就是由 C 语言开发的。

7.1.1 项目

下载 lab7.rar 文件后，你将看到以下的结构目录:

- cmake-build-debug

- CMakeLists.txt
- Lab7.cpp

7.1.2 学习内容

Lab7 的学习内容相对来说是最为庞大的，不仅面对的是一份轻量的工程化代码，而且更多的需要是花时间来理解整个流程。

- 树的递归
- 字符串的操作
- 阅读代码
- 理解解释器运作过程
- 类型转换
- 测试案例编写

7.1.3 完成内容

你需要用 C++ 来进行 Python 语句的解析，同时能够实现实时交互与代码维护。

7.1.4 前置内容

Python 语言的学习

python 代码展示

文件 (F) 动作 (A) 编辑 (E) 查看 (V) 帮助 (H)

```
输出:
print(100) # 或者print 100 (取决于python版本)
a = 10 # 不需要声明变量的类型
b = 10
c = a + b # 进行加法运算
```

只需要了解这些语句即可，我们需要做的就是把这些不需要声明变量的语句，解析成 C++ 版本的语句, 从某种角度来说似乎就是字符串的读写，但工业级别的代码远没有想象中的简单。

7.2 Getting Started

开始前我们需要先看一看解释器的整个框架, 当然这里面有很多晦涩难懂的概念，我们需要一步一步来进行解释

7.2.1 解释器运作图

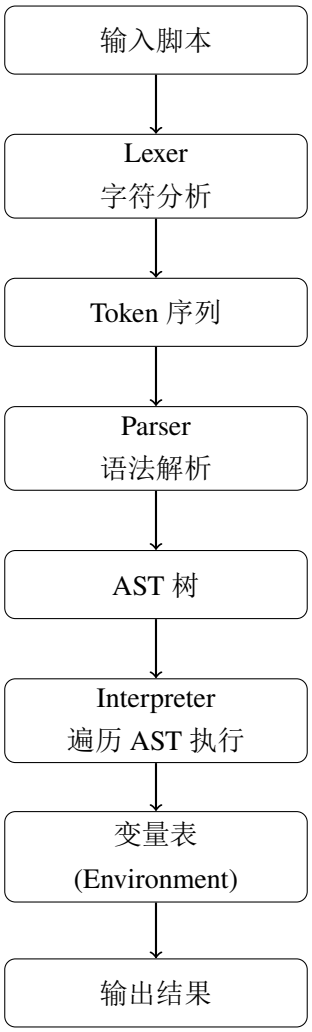


图 7.1: Python 子集解释器的整体执行流程

7.2.2 Phase 1

依旧是惊险而又刺激的读代码环节，我们首先需要了解解释器每个部位是怎么运作的

```
enum TokenType {
    TOKEN_INT,    // 整数
    TOKEN_IDENTIFIER, // 变量名
    TOKEN_ASSIGN, // '='
    TOKEN_PLUS,  // '+'
    TOKEN_MINUS, // '-'
    TOKEN_MUL,   // '*'
    TOKEN_DIV,   // '/'
    TOKEN_LPAREN, // '('
    TOKEN_RPAREN, // ')'
    TOKEN_PRINT, // 'print' 关键字
    TOKEN_SEMICOLON, // ';'
    TOKEN_EOF    // 文件结束
};
```

开场定义了一个 Token 的 enum 类型，分别存储了一些 Token 的类型，比如我们可能在分析过程中遇到，整数，加号，一些关键字等等

Definition 7.1

在编程语言的世界里，Token（标记/令牌）是指代码中不可再分的最小语法单位。如果把代码比作“句子”，那么 Token 就是句子中的“单词”。词法分析器（Lexer）的工作就是把一串长长的字符流，切分成一个个有意义的 Token。

然后用 Token 记录了这个变量的类型与大小，记住 Python 中是可以存储无限大的整数，是因为本身就是字符串类型存储

```
struct Token {
    TokenType type;
    string value;
    Token(TokenType t, string v = "") : type(t), value(v) {}
};
```

7.2.3 Phase 2

在该阶段我们需要实现一个 Lexer 类，这个类的作用是将语句分成一系列的 Token

- **构造函数**：初始化三个变量:
  - source: 读入的本文，比如 c = a + b;
  - position: 初始位置
  - currentChar: 当前位置的字符，请确保当前没有到整个语句结尾, 否则初始化为反斜杠 0;
- **advance()**：向前移动一个位置，请确保当前没有到整个语句结尾, 否则初始化为反斜杠 0;
- **skipWhiteSpace()**：如果是空格就向前移动一个位置

Axiom 7.1 (Hint)

- 可以使用 *isspace* 函数来判断是否为空格
- 可以使用 *advance()* 来向前遍历

- **parseInteger()**：如果当前位置是数字就开始解析整数，返回值是一个字符串，防溢出。

Axiom 7.2 (Hint)

- 可以使用 *isdigit* 函数来判断是否为空格
- 可以使用 *advance()* 来向前遍历

- **parseIdentifier()**：解析标识符，如果当前位置是数字字母或者下划线，就是一个标识符。

Axiom 7.3 (Hint)

- 可以使用 *isalnum* 函数来判断是否为字母 + 数字
- 可以使用 *advance()* 来向前遍历

最后实现一个 getNextToken() 函数，也是最重要，最难的一个函数。

该函数需要返回一个 Token 类型，目的是当我们通过 parse 解析到一个小的整数 Token 或者标识符 Token 的时候，我们需要返回这个 Token 的类型，比如如果我们解析到的是乘号，请返回 Token(TOKEN_MUL)，与最初提到的 Tokentype 类型表对应，如果是数字还需要返回它的大小（字符串类型），尤其注意需要解析 print。

有效代码：20 lines 左右

由于项目庞大，我们这里贴心的为大家准备了测试函数，因为对于普通的题目我觉得大家是没有写测试案例的习惯的，大家只会一味地去写但是不去看看自己写对了没，没写对就是去问 AI，但是对于一个工程项目，需要干的是去写测试案例。

Definition 7.2 (测试驱动开发)

在编程开发中，TDD (Test-Driven Development) 即测试驱动开发，是一种非常有效的设计方法论。它的核心思想是：在编写功能代码之前，先编写测试代码。

友情提示

测试驱动开发是一个很重要的东西，我也是在开始写 Java 之后意识到的，但是计算机小学期的老师摆烂加上学生一心追逐分数，也造成了这个核心思想的埋没。

用一个话来讲就是：我们经常要把作业交到 CG 上，但是为什么我的代码明明有不严谨的地方为什么却过了，这就是因为后台测试的数据太弱了，那为什么数据一加强，我的代码就过不了了，是因为我的代码太弱了。发现没有: 这是一个互促的过程，不断地提高自己的测试用例强度，不断完善代码细节部分。

我们这里准备的测试案例 testLexer()(记得注释掉 testOJ()) 算比较弱的，鼓励大家编写更强的，编写过程会放到下一个 Phase

7.2.4 Phase 3

学习抽象语法树，即 AST 树，那么首先要了解什么是树。

Definition 7.3 (树)

树是 n（$n>=0$）个结点的有限集。当 $n=0$ 时，称为空树。在任意一棵非空树中应满足：

- 有且仅有一个特定的称为根的结点.
- 当 $n>1$ 时，其余节点可分为 m（$m>0$）个互不相交的有限集 $T1,T2,\cdots,Tm$，其中每个集合本身又是一棵树，并且称为根的子树。

显然，树的定义是递归的，即在树的定义中又用到了自身，树是一种递归的数据结构。树作为一种逻辑结构，同时也是一种分层结构，具有以下两个特点：

- 树的根结点没有前驱，除根结点外的所有结点有且只有一个前驱。
- 树中所有结点可以有零个或多个后继。

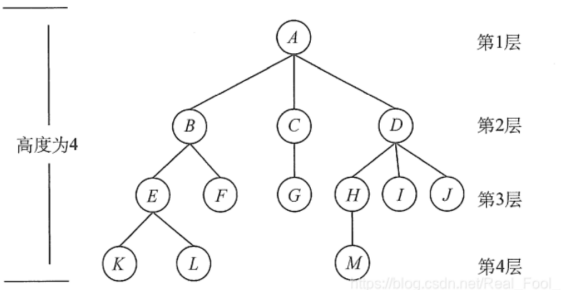


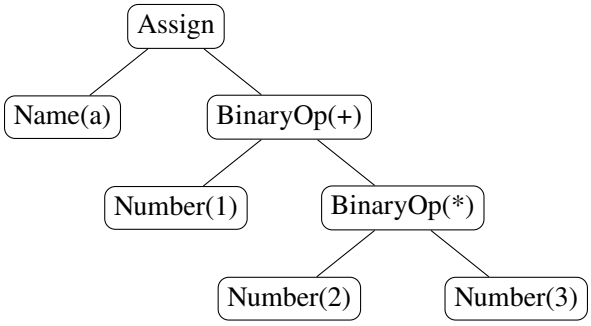
图 7.2: 树的结构

一棵树有自己的左儿子和右儿子，我们可以看一下在 python 解释器中的具体形式

如果你只用“字符串”来执行代码，会遇到三个致命问题：

- 运算优先级难处理 1 + 2 * 3
- 结构信息丢失 if a : print(a)
- 分析、优化、执行全混在一起

如果用一棵树存起来



我们可以从下往上算，一个父节点就代表着要运算的符号，下面的代码可以理解成定义了这些节点，我们不关注是如何实现这课树的，只需要知道是这样存起来运算符的即可

```
struct ASTNode { virtual ~ASTNode() {} };

struct BinaryOpNode : ASTNode {
    ASTNode *left, *right;
    Token op;
    BinaryOpNode(ASTNode* l, Token o, ASTNode* r) : left(l), op(o), right(r) {}
    ~BinaryOpNode() { delete left; delete right; }
};

struct IntegerNode : ASTNode {
    int value;
    IntegerNode(int v) : value(v) {}
};

struct VariableNode : ASTNode {
    string name;
    VariableNode(const string& n) : name(n) {}
};

struct AssignmentNode : ASTNode {
    string name;
    ASTNode* expr;
    AssignmentNode(const string& n, ASTNode* e) : name(n), expr(e) {}
    ~AssignmentNode() { delete expr; }
};

struct PrintNode : ASTNode {
    ASTNode* expr;
    PrintNode(ASTNode* e) : expr(e) {}
    ~PrintNode() { delete expr; }
};
```

可能唯一需要知道的就是有 Print 节点，赋值语句节点，变量节点，整数节点，二元运算符节点。

7.2.5 Phase 4

这个阶段我们要实现将分析出来的 Token 转换成一颗语法树，请时刻记住，整棵树用的是递归方法。定义一个 Parset 类，这个类的作用就是构建这棵树，这个类有两个变量以及一个私有成员函数

```
Lexer& lexer;
Token currentToken;
void eat(TokenType t) {
    if (currentToken.type == t) currentToken = lexer.getNextToken();
    else exit(1);
}
```

- lexer：存储一些列的 Token
- currentToken：当前要构建的 Token
- eat() 验证“当前 token 是否符合预期”，，可以理解成本来应该是 a + b , 你写成了 a b + , 这就是非法 Token 序列。

接下来就是我们需要构建的:

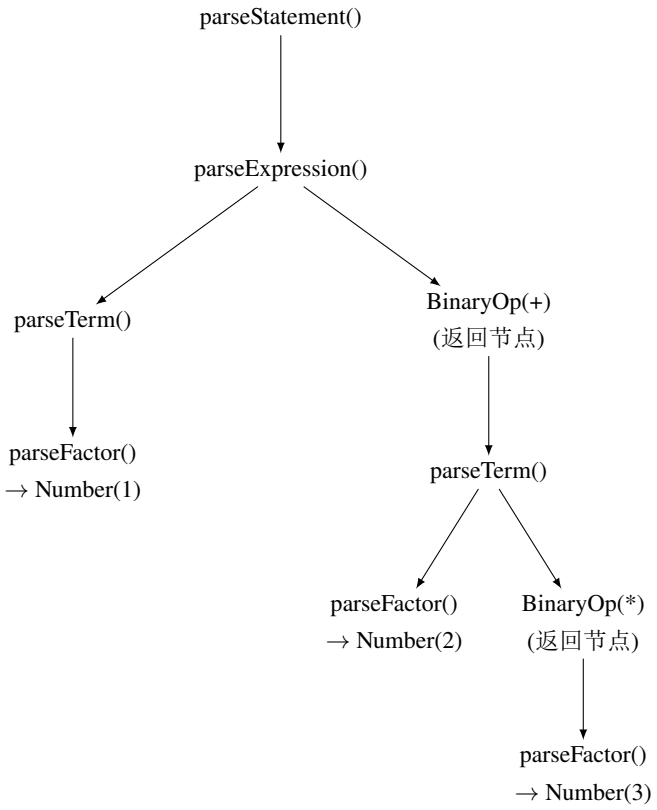
- **parseFactor()**：用于解析最小原子，就是不可再分的，可能是整数，可能是左括号，可能是右括号等等，有点抽象，可以看看下面的代码

```
if (tok.type == TOKEN_INT) { eat(TOKEN_INT); return new IntegerNode(stoi(tok.value)); }
```

- **parseExpression()**：Term 是由 factor 通过高优先级运算符组合而成的结构
- **parseTerm()**:Expression 是最外层的算术结构，向内处理低优先级运算符。

这里想了想，三个函数是相互牵连的，整体讲一下：可以下面的函数调用图，其中 Number(1), 以及一些节点就是我们要构架的

函数调用



案例描述如下:

对于解析完的 Token 序列 [INT(1), PLUS, INT(2), MUL, INT(3), EOF]

1. 从 parseExpression 开始（顶层入口）,expression 永远先让 term 把“该先算的”处理完，所以 expression 需要调用 Term()
2. 解析 Term 前需要解析 Factor(), 用来解析每个原子，比如 Number 或者 Op
3. 调用 parseFactor()
   - 当前 token：INT(1)
   - 识别整数
   - eat(INT)
   - 返回 Node(1)
4. 返回到 parseTerm():
   - node = 1
   - currentToken = PLUS
5. 当前是加号，不需要停留 Term，只需要返回即可，如果是乘号，就需要进一步考虑了
6. 回到 parseExpression(), 由于是加法调用 parseTerm 再拿右边，依次循环

每个函数有效代码: 7-12 行

开发的难度还是很大的，为了不加重大家的负担这里的 parseStatement() 就不再设置题

测试案例编写：

如何编写自己的测试案例呢？我们可以用 #include<cassert> 中的一个语句叫 assert

其实很简单

```
int a = 3;
int b = 4;
assert(a + b == 7);
```

如果这句话，不为真那么程序就会中断了，那么我们可以通过前面的解析器，当然我也提供了这个测试函数的编写，但是如果你不想写，请读懂测试函数怎么写的, 因为我的测试太弱了。

7.2.6 Phase 5

构建最终的解释器类，现在把一句语句拆成了 Token，然后又解析了算术表达式顺序，接下来就是解释这个答案是多少了。

说实话这是最好写的一个类了，只需要判断节点类型，然后返回值。(写到这里助教已经开始发蒙了)

最终实现 int evaluate(ASTNode* node)，这个函数有一个参数

- 一个 ASTNode*，即某条语句或表达式的 AST 根节点

这里介绍一个关键字，是在第一学期上机课提到的叫 dynamic_cast, 这里用一段代码来解释

```
auto n = dynamic_cast<IntegerNode*>(node)
```

解释的时候情况如下：

1. 整数节点返回值
2. 变量节点，通过哈希表返回对应的变量值
3. 二元操作节点，通过左右节点，以及当前节点的运算符，来返回具体值
4. Print 节点，输出之后返回节点的 expr

7.3 submit

7.3.1 提交地点和方式

请将你的最终代码文件提交到 cg 平台，提交 cpp 后缀的文件，同时将模式切换到 testOJ(), 其他测试注释掉.

7.3.2 credit

请确保所有文件均为自己编写，遇到不会的可以去查询，但是切勿抄袭他人成果，只有自己努力去写了，不会的去问了，才能真正学到很多知识。

7.4 References

- 树
- python 中的 AST 树
- C++ 断言语句